

Módulo: Programación Multimedia y dispositivos móviles. Unidad de Trabajo Nº1: Introducción al desarrollo de aplicaciones móviles. Actividad de Clase 4: Análisis y modificación de Aplicación móvil legacy.

Análisis de la Aplicación

Realiza lo siguiente:

- 1) Descarga el archivo con el Proyecto en Android Studio. ✓
- 2) Descomprimir dicho archivo en el directorio donde tengas tu Proyectos de Android Studio. ✓
- 3) Abre dicho Proyecto. Ejecútalo y comprueba su funcionamiento. ✓
- 4) Accede al archivo **AndroidManifest.xml** y registra en un documento las principales opciones que tiene dicho fichero. ¿Cuántas activities tiene la aplicación?

Esta aplicación contiene 2 activities, que una de ellas es el main y la segunda. Esto quiere decir que vamos a tener dos pantallas diferentes.

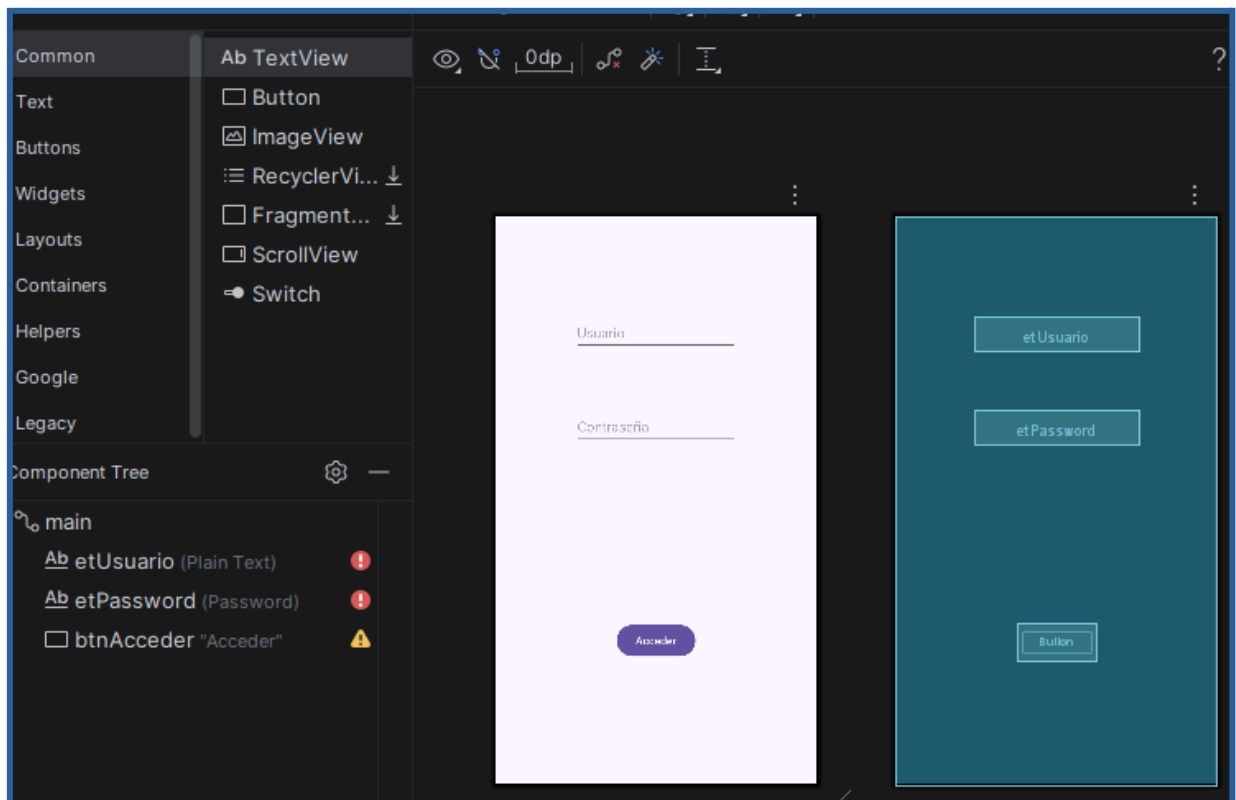
```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.AplicacionVistas">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:windowSoftInputMode="adjustResize">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity
            android:name=".Segunda"
            android:exported="false"
            android:windowSoftInputMode="adjustResize" />
    </application>
```

- 5) Abre el archivo principal de la interfaz de usuario (usualmente **activity_main.xml**) e identifica las clases de vistas principales que componen la pantalla, como **TextView**, **Button** o **EditText**. Registra en el documento qué elementos visuales controla cada clase y cómo están organizados.

En este archivo tenemos las siguientes vistas:

- **EditText** permite introducir texto. En concreto, esta aplicación tiene dos, uno para escribir el usuario y otro para la contraseña, que oculta los caracteres.
- **Button** representa el botón «Acceder». Lo que nos permite que al pulsarlo se ejecute la acción que tenga asociada.
- **ConstraintLayout** es el contenedor donde están estos elementos. Se encarga principalmente de colocarlos en la pantalla mediante diversas restricciones.

En cuanto a la organización, el campo de usuario está arriba, el de contraseña debajo y el botón «Acceder» más abajo, alineados aproximadamente en el centro de la pantalla.



6) Abre también la clase de Kotlin asociada a dicho archivo xml e identifica en el código como se accede a los componentes y qué y cómo funciona la aplicación.

Estructura de los componentes:

1. El método `onCreate` se utiliza `setContentView` que indica que se muestra en `MainActivity` es la definida en `activity_main.xml`.
2. Por otro lado, se accede a los componentes del XML a través del `findViewById`, de esta forma el código puede trabajar con los elementos definidos en el XML.
3. Para el botón se establece un listener con el que nos permite crear un objeto `Intent` cuando se pulsa, este objeto sirve para comunicarse con otra actividad, en este caso desde el main hasta la segunda.
4. Antes de cambiar de pantalla, se obtiene el texto introducido en el campo de usuario(`etUsuario`), por tanto `putExtra` permite enviar información adicional a la siguiente actividad y la contraseña se obtiene mediante `etPassword` pero en el código no se envía a la segunda actividad.

```
// class MainActivity : AppCompatActivity() {
//     override fun onCreate(savedInstanceState: Bundle?) {
//         super.onCreate(savedInstanceState)
//         setContentView(R.layout.activity_main)
//         //Accedemos a los componentes que definimos en fichero xml asociado a esta Activity.
//         val etUsuario = findViewById<EditText>(R.id.etUsuario)
//         val etPassword = findViewById<EditText>(R.id.etPassword)
//         val btnAcceder = findViewById<Button>(R.id.btnAcceder)
//         //Definimos el listener para el boton acceder
//         btnAcceder.setOnClickListener {
//             //Creamos el objeto Intent para comunicarnos con la otra Activity
//             val intentol = Intent(packageContext, this@MainActivity, 0, Segunda::class.java)
//             //Pasamos el parametro en este caso el nombre de usuario.
//             intentol.putExtra(name = "usuario", value = etUsuario.text.toString())
//             //Iniciamos la actividad
//             startActivity(intentol)
//         }
//     }
// }
```

5. Para el inicio de la segunda actividad, se usa el `startActivity`, que a su vez ejecuta el intent.

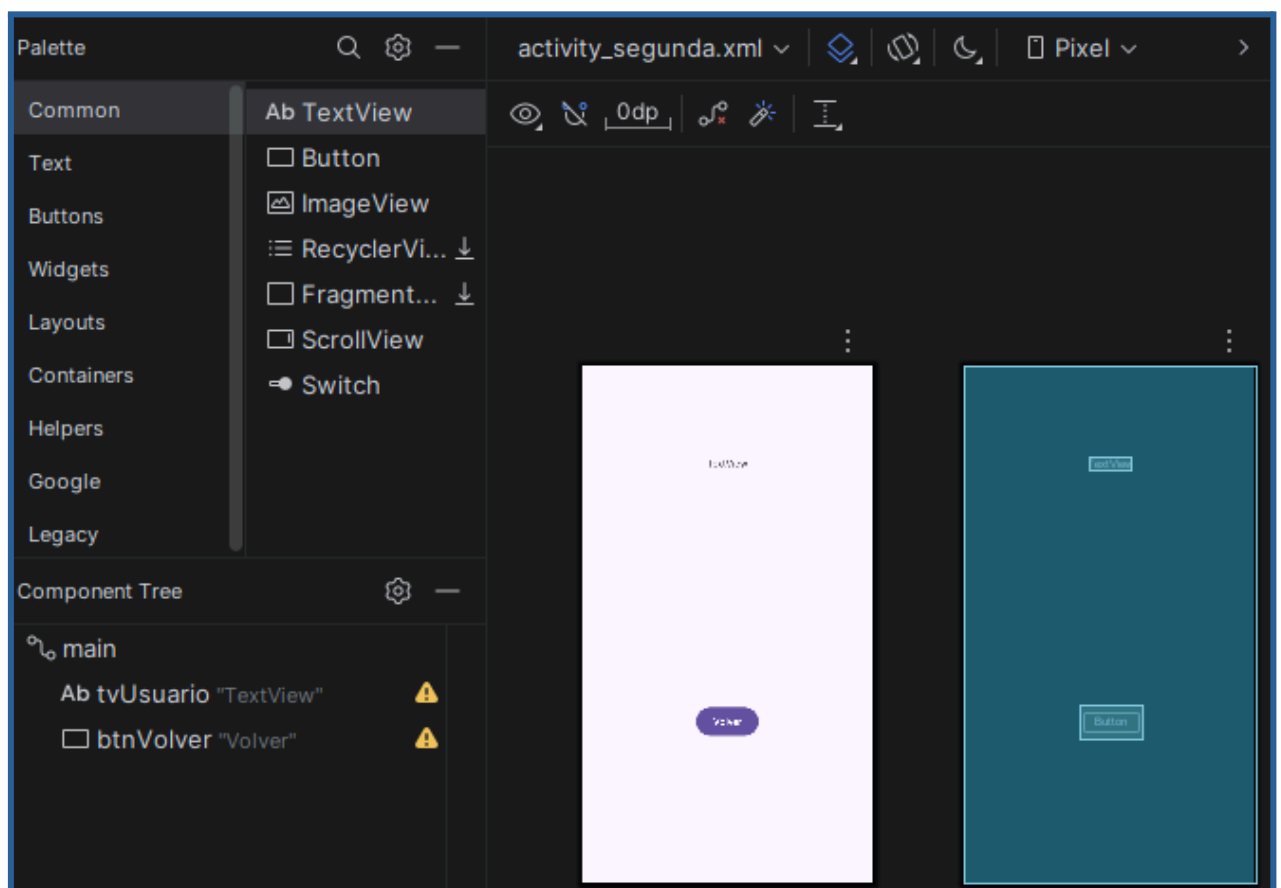
Por lo que el funcionamiento de la aplicación quedaría de la siguiente forma, el `MainActivity` actúa como pantalla de inicio de sesión. Recupera los componentes del XML mediante `findViewById`, espera a que se pulse el botón `Acceder` y, al hacerlo, crea un `Intent` para abrir la segunda actividad, enviándole el nombre de usuario introducido.

7) En caso de que la aplicación tenga más actividades abre el fichero donde está la interfaz de usuario que tiene. identifica las clases de vistas principales que componen la pantalla, como `TextView`, `Button` o `EditText`. Registra en el documento qué elementos visuales controla cada clase y cómo están organizados.

En este archivo tenemos las siguientes vistas:

- Tenemos un `TextView` llamado `tvUsuario`, que se utiliza para mostrar el nombre del usuario que se ha introducido en la pantalla anterior.
- Tenemos un `Button` llamado `btnVolver`, que corresponde al botón «Volver» y permite ejecutar la acción asociada cuando el usuario lo pulsa.
- `ConstraintLayout` como el en el caso anterior es el contenedor donde están estos elementos.

En cuanto a la organización, el `TextView tvUsuario` se encuentra en la parte superior, aproximadamente en el centro de la pantalla, y debajo se encuentra el `Button btnVolver`, también centrado horizontalmente.



8) Abre también la clase de Kotlin asociada al último archivo xml que acabas de abrir e identifica en el código como se accede a los componentes y qué y cómo funciona la aplicación.

Estructura de los componentes:

1. El método onCreate se utiliza junto con setContentView para indicar que la interfaz que se muestra en Segunda es la definida en activity_segunda.xml.
2. Por otro lado, se accede a los componentes del XML a través de findViewById, de esta forma el código puede trabajar con los elementos definidos en el XML, en este caso el TextView tvUsuario y el Button btnVolver.
3. Para obtener el usuario que se ha enviado desde la otra Activity se utiliza un objeto Bundle, que permite acceder a los datos enviados mediante el Intent. Se obtiene el valor utilizando la clave "usuario".
4. Una vez obtenido el usuario, se modifica el texto del TextView mediante tvusuario.text, mostrando un mensaje como «Hola [usuario]». De esta forma, aparece en la segunda pantalla el nombre introducido anteriormente.
5. Para el botón «Volver» se establece un listener. Cuando se pulsa, se crea un nuevo Intent que permite comunicarse con la MainActivity y volver a la pantalla principal de la aplicación.

```
class Segunda : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_segunda)
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
            val systemBars = insets.getInsets(typeMask = WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
            insets
        }
        //Definimos un objeto Bundle para acceder a los datos que se han enviado desde la otra Activity
        val bundle = intent.extras
        val usuario = bundle?.getString(key = "usuario")

        //Accedemos a los componentes que definimos en fichero xml asociado a esta Activity.
        val tvusuario = findViewById<TextView>(R.id.tvusuario)
        val btnvolver = findViewById<Button>(R.id.btnvolver)

        tvusuario.text = "Hola $usuario"

        //Definimos el listener para el boton volver
        btnvolver.setOnClickListener {
            //Creamos el objeto Intent para comunicarnos con la Activity Principal
            val intento2 = Intent(packageContext, this@Segunda, cls = MainActivity::class.java)
            //Iniciamos la actividad
            startActivity(intento2)
        }
    }
}
```

En resumen, la segunda activity recibe el nombre de usuario enviado desde la pantalla anterior y lo muestra en un TextView mediante un mensaje de bienvenida. Además, dispone de un botón «Volver» que permite regresar a la MainActivity.

Modificación de la Aplicación

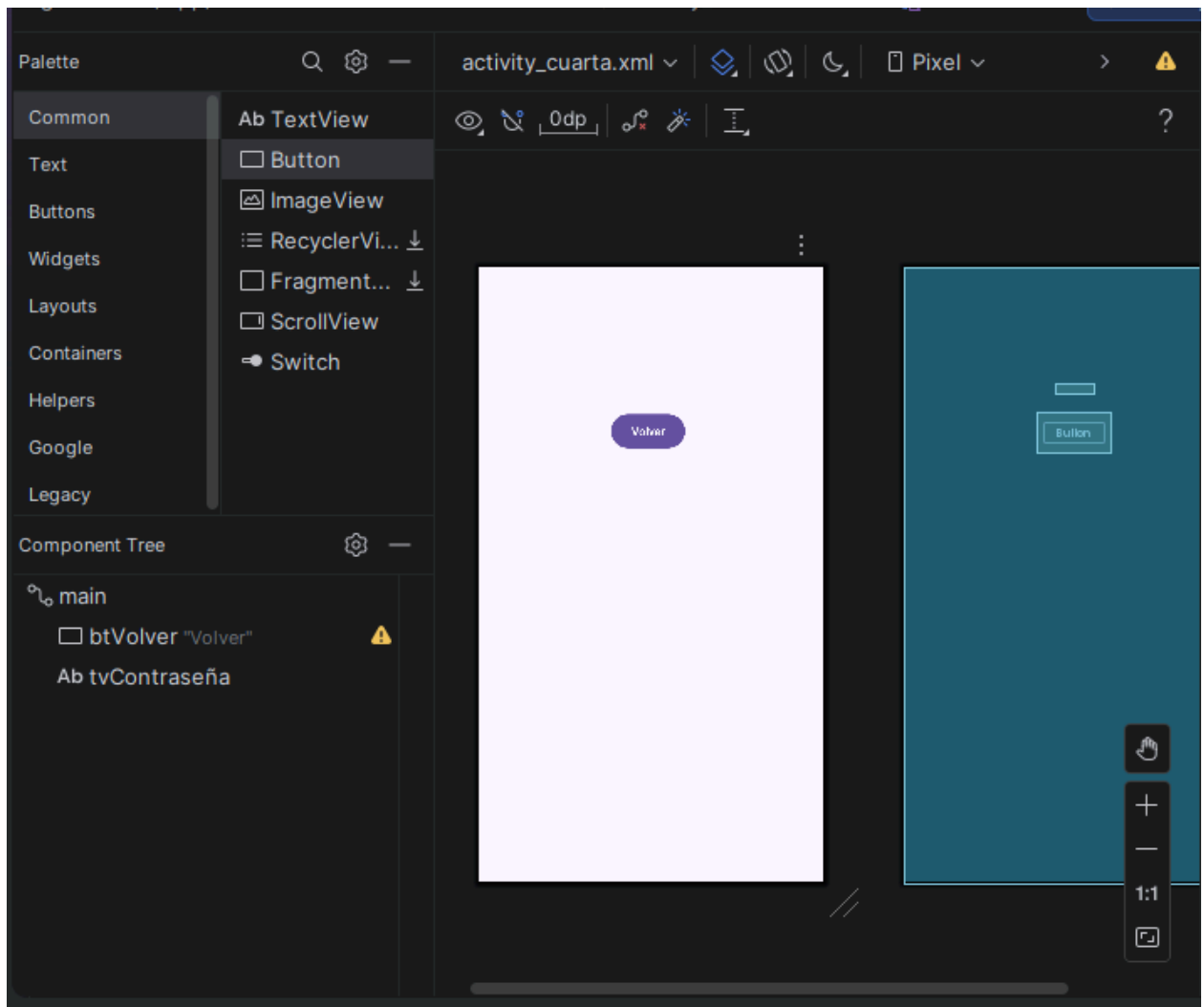
9) Modifica el archivo XML (usualmente activity_main.xml) y agrega una nueva funcionalidad visual. Por ejemplo, añade un nuevo Button en la parte inferior de la pantalla que al pulsarlo abra otra nueva activity (ventana de la aplicación).

He creado una nueva Activity llamada Tercera, que es la pantalla que se abre al pulsar el nuevo botón de la pantalla principal. En el XML he añadido un TextView para mostrar la información de la tercera pantalla y un Button para poder acceder a ella desde la principal.

```
btnperfil.setOnClickListener {
    val intent = Intent(packageContext, this@MainActivity, cls = Tercera::class.java)
    startActivity(intent)
}
```

10) Crea la nueva activity que será la ventana que se abra cuando pulse el botón que acabas de añadir. Modifica el archivo XML que acabas de crear añadiendo un TextView donde muestres la contraseña y un Button que al pulsarlo volverá a la principal.

Aquí en la cuarta activity añadí los componentes y los nombre con su función para poder distinguirlos después en el código.



11) En la clase de Kotlin asociada al archivo xml que acabas de crear. Donde accedas a la contraseña y la muestres y que el usuario pulse el botón la aplicación vuelva a la activity principal.

Conecté las diferentes Activities para que la contraseña introducida en MainActivity se pueda pasar hasta Cuarta. Desde Tercera, el botón btnAccederPerfil abre la cuarta pantalla y le pasa la contraseña. En Cuarta se muestra la contraseña mediante un TextView y el botón btVolver permite regresar a la pantalla principal

```
class Cuarta : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        ViewCompat.setOnApplyWindowInsetsListener(view = findViewById(R.id.main)) { v, insets ->
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
            insets
        }

        // Recibimos la contraseña de la Activity anterior
        val password = intent.getStringExtra(name = "password")
        // Accedemos a los componentes del XML
        val tvcontrasena = findViewById<TextView>(R.id.tvContraseña)
        val btvolver = findViewById<Button>(R.id.btVolver)
        // Mostramos la contraseña
        tvcontrasena.text = "Tu contraseña es: $password"
        // Volvemos a la pantalla principal
        btvolver.setOnClickListener {
            val intent = Intent(packageContext = this@Cuarta, cls = MainActivity::class.java)
            startActivity(intent)
        }
    }
}
```

12) Ejecuta la aplicación y verifica que la nueva funcionalidad funciona correctamente .

